

Competitive Programming

Section 19 Editorial: Dijkstra And Weighted Modeling

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	3
2	Dijkstra reminder	4
3	A. Weighted Route Queries	5
3.1	Restatement	5
3.2	Observations	5
3.3	Algorithm	5
3.4	Correctness proof	5
3.5	Complexity	5
3.6	Full C++ solution	6
3.7	Full Python solution	7
4	B. Grid Toll Path	8
4.1	Restatement	8
4.2	Observations	8
4.3	Correctness	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	9
5	C. Discount Route	11
5.1	Restatement	11
5.2	Observations	11
5.3	Correctness	11
5.4	Complexity	11
5.5	Full C++ solution	11
5.6	Full Python solution	12
6	Common wrong answers	14
7	Stress testing	15
8	What to carry forward	16

1 Editorial overview

Problem	Model
A. Weighted Route Queries	directed graph Dijkstra
B. Grid Toll Path	grid cells as weighted graph
C. Discount Route	two-layer state graph

2 Dijkstra reminder

Dijkstra maintains tentative distances and always expands the smallest tentative distance first. With nonnegative weights, the first non-stale pop of a state finalizes its shortest distance.

Stale heap entries happen because we push improved distances instead of decreasing keys:

```
if popped_distance != dist[state]:  
    skip it
```

3 A. Weighted Route Queries

3.1 Restatement

Given a directed weighted graph, source s , and queries, print the shortest distance from s to each queried vertex or -1 .

3.2 Observations

All queries share the same source, so run Dijkstra once and answer queries by array lookup.

3.3 Algorithm

```
build adjacency list
run Dijkstra from s
for each query x:
    print dist[x] or -1
```

3.4 Correctness proof

Lemma. When Dijkstra pops a non-stale pair for vertex u , $\text{dist}[u]$ is the shortest distance from s to u .

Proof. Suppose a shorter path to u existed. Let x be the first vertex on that path not finalized yet, and p its predecessor. Vertex p was finalized earlier, so edge $p \rightarrow x$ would have relaxed x to at most the shorter path prefix. Since all weights are nonnegative, that distance would be no larger than the alleged shorter distance to u , contradicting that u was the smallest popped state. $\square$

Theorem. Every printed query answer is correct.

Proof. By the lemma, Dijkstra computes every reachable shortest distance. Unreached vertices keep the sentinel value and are correctly printed as -1 . $\square$

3.5 Complexity

```
Time:  $O((n + m) \log n)$ 
Memory:  $O(n + m)$ 
```

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s, qn;
    if (!(cin >> n >> m >> s >> qn)) return 0;
    --s;
    vector<vector<pair<int, long long>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v; long long w;
        cin >> u >> v >> w;
        adj[u - 1].push_back({v - 1, w});
    }
    vector<int> queries(qn);
    for (int &x : queries) { cin >> x; --x; }

    const long long INF = (1LL << 62);
    vector<long long> dist(n, INF);
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<pair<long long, int>>> pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            long long nd = d + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                pq.push({nd, v});
            }
        }
    }

    for (int i = 0; i < qn; i++) {
        if (i) cout << ' ';
        cout << (dist[queries[i]] == INF ? -1 : dist[queries[i]]);
    }
    cout << '\n';
    return 0;
}
```

3.7 Full Python solution

```
import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it); s = next(it) - 1; qn = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1; v = next(it) - 1; w = next(it)
        adj[u].append((v, w))
    queries = [next(it) - 1 for _ in range(qn)]

    inf = 10**30
    dist = [inf] * n
    dist[s] = 0
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d != dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    print(" ".join(str(-1 if dist[x] == inf else dist[x]) for x in queries))

if __name__ == "__main__":
    main()
```

4 B. Grid Toll Path

4.1 Restatement

Find the minimum total toll from the top-left cell to the bottom-right cell. Entering a cell costs that cell's toll, and the start cell is included.

4.2 Observations

Each cell is a graph vertex. Moving to a neighboring cell is a directed edge whose cost is the neighbor's toll. Costs are nonnegative, so Dijkstra applies.

4.3 Correctness

Dijkstra's invariant applies directly to cell states. Every legal grid walk is a path in the graph, and every graph path is a legal grid walk. Therefore the shortest graph distance to $(n-1, m-1)$ is exactly the minimum toll path.

4.4 Complexity

```
Time:  $O(n*m*\log(n*m))$   
Memory:  $O(n*m)$ 
```

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<long long>> cost(n, vector<long long>(m));
    for (auto &row : cost) for (long long &x : row) cin >> x;
    const long long INF = (1LL << 62);
    vector<vector<long long>> dist(n, vector<long long>(m, INF));
```

```

using State = tuple<long long,int,int>;
priority_queue<State, vector<State>, greater<State>> pq;
dist[0][0] = cost[0][0];
pq.push({dist[0][0], 0, 0});
int dr[4] = {1, -1, 0, 0};
int dc[4] = {0, 0, 1, -1};
while (!pq.empty()) {
    auto [d, r, c] = pq.top(); pq.pop();
    if (d != dist[r][c]) continue;
    for (int i = 0; i < 4; i++) {
        int nr = r + dr[i], nc = c + dc[i];
        if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
        long long nd = d + cost[nr][nc];
        if (nd < dist[nr][nc]) {
            dist[nr][nc] = nd;
            pq.push({nd, nr, nc});
        }
    }
}
cout << dist[n - 1][m - 1] << '\n';
return 0;
}

```

4.6 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it)
    cost = [[next(it) for _ in range(m)] for _ in range(n)]
    inf = 10**30
    dist = [[inf] * m for _ in range(n)]
    dist[0][0] = cost[0][0]
    heap = [(cost[0][0], 0, 0)]
    while heap:
        d, r, c = heapq.heappop(heap)
        if d != dist[r][c]:
            continue
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m:
                nd = d + cost[nr][nc]

```

```
        if nd < dist[nr][nc]:
            dist[nr][nc] = nd
            heapq.heappush(heap, (nd, nr, nc))
    print(dist[n - 1][m - 1])

if __name__ == "__main__":
    main()
```

5 C. Discount Route

5.1 Restatement

Find the cheapest path from 1 to n when one edge may be discounted to $\text{floor}(w/2)$.

5.2 Observations

The vertex alone is not enough; we must know whether the discount was already used. Use states:

```
(u,0): at u, discount unused
(u,1): at u, discount used
```

All expanded edges have nonnegative weights, so Dijkstra works.

5.3 Correctness

Every valid original route corresponds to one path in the expanded graph: normal edges stay in the same layer, and the discounted edge moves from layer 0 to layer 1. Conversely, every expanded-graph path describes a valid route using the discount at most once. Dijkstra therefore finds the optimal route.

5.4 Complexity

```
Time:  $O((n + m) \log n)$ 
Memory:  $O(n + m)$ 
```

5.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
```

```

if (!(cin >> n >> m)) return 0;
vector<vector<pair<int, long long>>> adj(n);
for (int i = 0; i < m; i++) {
    int u, v; long long w;
    cin >> u >> v >> w;
    adj[u - 1].push_back({v - 1, w});
}
const long long INF = (1LL << 62);
vector<array<long long, 2>> dist(n, {INF, INF});
using State = tuple<long long, int, int>;
priority_queue<State, vector<State>, greater<State>> pq;
dist[0][0] = 0;
pq.push({0, 0, 0});
while (!pq.empty()) {
    auto [d, u, used] = pq.top(); pq.pop();
    if (d != dist[u][used]) continue;
    for (auto [v, w] : adj[u]) {
        long long nd = d + w;
        if (nd < dist[v][used]) {
            dist[v][used] = nd;
            pq.push({nd, v, used});
        }
        if (!used) {
            nd = d + w / 2;
            if (nd < dist[v][1]) {
                dist[v][1] = nd;
                pq.push({nd, v, 1});
            }
        }
    }
}
long long ans = min(dist[n - 1][0], dist[n - 1][1]);
cout << (ans == INF ? -1 : ans) << '\n';
return 0;
}

```

5.6 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it)

```

```

adj = [[] for _ in range(n)]
for _ in range(m):
    u = next(it) - 1; v = next(it) - 1; w = next(it)
    adj[u].append((v, w))
inf = 10**30
dist = [[inf, inf] for _ in range(n)]
dist[0][0] = 0
heap = [(0, 0, 0)]
while heap:
    d, u, used = heapq.heappop(heap)
    if d != dist[u][used]:
        continue
    for v, w in adj[u]:
        nd = d + w
        if nd < dist[v][used]:
            dist[v][used] = nd
            heapq.heappush(heap, (nd, v, used))
        if used == 0:
            nd = d + w // 2
            if nd < dist[v][1]:
                dist[v][1] = nd
                heapq.heappush(heap, (nd, v, 1))
ans = min(dist[n - 1])
print(-1 if ans == inf else ans)

if __name__ == "__main__":
    main()

```

6 Common wrong answers

- Using BFS because the graph is “just shortest path”.
- Marking a vertex visited when it is pushed into the heap.
- Forgetting to skip stale heap entries.
- Using 32-bit `int` for distances.
- Applying Dijkstra to negative edges.
- In Discount Route, allowing the discount more than once.

7 Stress testing

For small graphs, compare Dijkstra to Floyd-Warshall. For grids, compare to Floyd-Warshall on the explicit small cell graph or to a slow repeated relaxation. For Discount Route, build the two-layer graph explicitly and run a generic Dijkstra to compare against the optimized state implementation.

8 What to carry forward

Dijkstra is the default shortest-path algorithm for nonnegative weighted graphs. The modeling skill is deciding what a state must remember. Once the state graph has nonnegative edge weights, the same invariant and heap template apply.